Assignment 2: Coding Basics

Yike Ma

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change "Student Name" on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.

2. Compute the mean and median of this sequence.

3. Ask R to determine whether the mean is greater than the median.

4. Insert comments in your code to describe what you are doing.

```
#1.
s1 <- seq (1,55,5)
#I generated the sequence and saved it as s1, which is convenient for following calculation

#2.
mean1 <- mean(s1)
median1 <- median(s1)

#3.
mean1 > median1
```

```
## [1] FALSE
```

```
# I compared mean value and median value, and returned as FALSE in the Console, which means the mean is
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```
v1 <- c("A", "B","C","D") #character
v2 <- c(88,68,77.5,49) #numeric
v3 <- c(FALSE, TRUE, TRUE, FALSE) #logic

df_student_information <- data.frame("names" = v1, "scores" = v2, "scholarship" =v3)
```

9. QUESTION: How is this data frame different from a matrix?

   Answer: Matrix is composed of numeric data, and can be used for numerical calculation. While data frame is a list of vectors with same length, which contains different types of data and can store information.

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word "Pass"; otherwise print the word "Fail".
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead if `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

```
#10. Create a function using if...else
f1 <- function(x){
  if(x > 50) {
  "Pass"
}
else {
  "Fail"}
}
#11. Create a function using ifelse()
f2 <- function(x){
  ifelse(x>50, "Pass", "Fail")
}
#12a. Run the first function with the value 52.5
f1(52.5) #"Pass"
```

```
## [1] "Pass"
```

```
#12b. Run the second function with the value 52.5
f2(52.5) #"Pass"
```

```
## [1] "Pass"
```

```r
#13a. Run the first function with the vector of test scores
#f1(v2) #"Error in if (x > 50) { : the condition has length > 1"
#13b. Run the second function with the vector of test scores
f2(v2) #"Pass" "Pass" "Pass" "Fail"
```

```
## [1] "Pass" "Pass" "Pass" "Fail"
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for "R vectorization")

    Answer: Ifelse worked. Ifelse() is a vectorized function, meaning all values in the vector can be calculated at the same time, instead of creating a loop to calculate one by one.

NOTE Before knitting, you'll need to comment out the call to the function in Q13 that does not work. (A document can't knit if the code it contains causes an error!)